

Introduction to Information Leakage Attacks

2025

1 Introduction

Cryptography, at its' core, is a discipline that connects theory with practice. After all, it is “easy” to construct an information theoretically secure (thus “unbreakable”) encryption scheme, e.g. the one-time pad. You may recall that the one-time pad simply exclusive-ors plaintext and key (we assume that there is some suitable binary representation of them). For the one-time pad we can prove information theoretic security assuming one chooses a one-time key uniformly at random, at the same length as the plaintext. Which in theory sounds easy, but in practice is not because not only could keys be very long (as they have to match the plaintext length), you also have to transmit this key information securely (i.e. it needs to stay secret and you need to authenticate the recipient). If you had a secure channel by which you could transmit the key, you could as well use it to transmit the message directly.

Thus in practical cryptography we always seek a trade-off between security and practicality. For instance, rather than generating truly random key material freshly for each message, we derive randomly looking key material, and we reuse it (for several blocks of a message, i.e. we use block ciphers). We also have to find a way to exchange keys over public channels (in the absence of “magic” channels) using public key cryptography. Finally, nobody would expect end users to engage with any of these tasks, thus we need to translate the mathematical description of a cryptographic scheme into some executable code. This translation is not trivial: typically we cannot bring forward all of the assumptions made in the proof(s) that may accompany the original description of the scheme. Attacks that target an implementation of a cryptographic scheme, using information that is inadvertently created by the implementation, have turned out to be extremely powerful in practice. Perhaps surprisingly, one of the most widely implemented techniques to make implementation attacks on cryptographic implementation harder is based on secret sharing. In order to see why this works, and how it works, we will first look at the philosophy behind implementation attacks, and in particular differential input attacks, and then drill into them to see what makes them work “better” and what makes the work “worse” in practice. Then we look at secret sharing as a countermeasure.

We start off by taking a bit of time to look into why power and electromagnetic emanations are often data dependent. Then we take a high level look at implementation

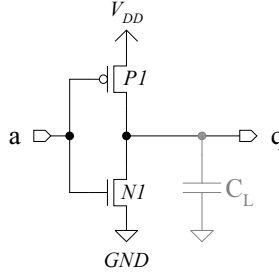


Figure 1: CMOS inverter.

attacks on the concrete example of RSA encryption. This example also enables us to briefly discuss reasons for data dependent timing information.

2 Power/EM information

The total power consumption of a device depends on the power consumption of all the logic cells and their interconnects making up the circuit. Hence, the total power consumption essentially depends on the number of logic cells in a circuit, the connections between them, and the technology based upon the cells are built. Thus architectural design (software and hardware), cell level and transistor level specificities all contribute to how “leaky” a circuit/device is.

Logic cells are very often implemented using CMOS¹. The discussion of the power consumption based on an inverter is representative for all other cells, because all CMOS cells are built based on complementary pull-up and pull-down networks (a network is just a fancy name for a bunch of connected gates). The inverter simply has two transistors $P1$ and $N1$, see Figure 1, whereas more complex gates just require more PMOS and NMOS transistors. In the inverter $P1$ is conducting and $N1$ is insulating if the input a is set to GND . Vice versa, $P1$ is insulating and $N1$ is conducting if the input a is set to V_{DD} . In both cases, there is no direct connection between the V_{DD} line and the GND line. All capacitances in the circuit are modelled as a single “capacitor” C_L .

The power consumption of the inverter (and therefore any other CMOS gate) is comprised of two components, see also Tab. 1. The first component is the static power consumption, which is the power that is consumed if there is no switching activity in a cell. The second component of the power consumption is the dynamic power consumption P_{dyn} , which is due to the switching of internal signals or output signals of a cell. The total power consumption of a cell is the sum of both static and dynamic power consumption. Typically, the dynamic power consumption is much more dominant than the static one.

¹Complementary Metal-Oxide-Semiconductor

However, in very small technologies this relationship is switched. The target device that we use in this unit is not based on a very small technology, hence the dynamic power consumption is dominant.

Power analysis attacks exploit the fact that the power consumption of cryptographic devices depends on the operations they perform and on the data they process. Hence it is useful to have a shorthand for these components when referring more mathematically to the power consumption. For each single power measurement, we refer to the operation-dependent component of the point as P_{op} , and to the data-dependent component as P_{data} . Besides P_{op} and P_{data} , each point of a power trace also depends on what is going on the other, possibly data and operation independent “stuff”, which we shall simply call “noise” P_{noise} . The components P_{op} , P_{data} , and P_{noise} are additive. Therefore, it is possible to simply model each point of a power trace as the sum of these components:

$$P_{total} = P_{op} + P_{data} + P_{noise}. \quad (1)$$

Now that we have a better sense of how the observable power consumption comes to happen, we can consider what a power trace actually is. Power traces are vectors of voltage values that have been recorded with a digital sampling oscilloscope (i.e. the Pico Scope in your coursework). The measured voltage values are proportional to the power consumption of a cryptographic device because the oscilloscope is connected to an appropriate measurement circuit or EM probe (in your case it is the former). The settings of the oscilloscope determine how many points are recorded per second (or clock cycle of the target device), as well as how many points are stored. The quality of the scope determines how big a range one has for both points per second as well as number of points. The quality of the probe and scope determine how much of the actual signal (i.e. P_{total}) is actually captured, how much this signal is influenced on its’ way from the probe to the scope, and thus what signal is actually available for digitisation on the scope. Thus the quality of a measured trace depends not only on the device, and the surroundings, but also every component that is part of the setup.

Table 1: Output transitions of the CMOS inverter and the corresponding power consumption.

Transition	Type of power consumption
0 → 0	static
0 → 1	static + dynamic
1 → 0	static + dynamic
1 → 1	static

3 Practical Attacks on RSA

The textbook RSA scheme works as follows. Given a key pair consisting of a public key pk and a secret key sk , with $pk = (e, N = pq)$ and $sk = d = e^{-1} \bmod \phi(N)$, the encryption of a plaintext m is given as $c = m^e \bmod N$. Decryption is given as $m = c^d \bmod (N)$.

3.1 Timing attacks

The attack that we are looking at briefly, is based on the fact that in many embedded systems, the RSA decryption is implemented using a so-called square-and-multiply algorithm, whereby the modular multiplications are implemented via the Montgomery method. Before proceeding to the attack, we briefly look at this implementation strategy.

3.1.1 Implementing decryption

Alg. A: RSA Decryption using the Binary Algorithm

The algorithm computes $c^d \bmod N$ using the left-to-right binary algorithm, and delivers m as return value.

```
input: int N, int d =  $(d_w, d_{w-1}, \dots, d_1, d_0)$ , int c  
output: int m  
begin  
   $m = 1$   
  for  $i = w - 1$  down to 0  
     $m = m \cdot m \bmod N$  // (Montgomery multiplication)  
    if  $d_i == 1$   
       $m = m \cdot c \bmod N$  // (Montgomery multiplication)  
  end  
  return m  
end
```

The binary square-and-multiply algorithm is given in Alg. A. The algorithm proceeds as follows: the algorithm works through the secret key d bit by bit. For each bit it performs a square (i.e. it computes $m \cdot m$). If and only if a key bit is equal to 1 it then also performs a multiply (i.e. it computes $c \cdot m$). These multiplications are not ordinary multiplications because in RSA all numbers live in $\mathbb{Z}_N$. Therefore one has to do modular multiplication, and a very common way of implementing this is by using the so-called Montgomery method. The outline of a Montgomery multiplication is given in Alg. B.

How and why this works takes a bit of time to explain, and understanding the working principle actually does not aid the understanding of attacks (or countermeasures) much.

The for-loop, and the various multiplications and additions, are in principle constant time. The last if statement in the algorithm however introduces some data dependency in terms of execution time. It checks the condition $r \geq N$, and then performs an extra operation if necessary. Therefore, for some inputs, the algorithm will have to do this extra operation. If we know that this algorithm is used, and we know things like N (which is public knowledge), and the base b (which can often be inferred from the architecture of the decryption device), then we can predict (given x and y) whether or not the extra step takes place during decryption.

Alg. B: Montgomery Multiplication

The algorithm computes the product of two numbers (in Montgomery form) modulo N .

```
input: int  $x, y$  with  $0 \leq x, y < N$ , int  $N$ , int  $b$ 
output: int  $r = x \cdot y \pmod{N}$ ,
 $r = 0$ 
for  $i = 0$  upto  $n-1$  do
   $u = (r_0 + x_i \cdot y_0) \cdot \omega \pmod{b}$ 
   $r = (r + x_i \cdot y + u \cdot N) / b$ 
end
if  $r \geq N$  then
   $r = r - N$ 
end
return  $r$ 
```

3.1.2 Differential timing attack

For the attack we assume that the adversary can perform the decryption multiple times with different inputs. It is not necessary that the adversary can choose the inputs, only that they know them. Our goal is to deduce the private key d by analysing timing information that we obtain by measuring response times of our decryption oracle, and by using a “model” of the decryption oracle². This attack setting is the “canonical” example for all side channel attacks on crypto implementations: given access to some implementation/device that performs an operation using a secret key, observe inputs (or outputs) and some associated side channel, in order to **recover the key**.

The attacker proceeds as follows. They randomly sample some/sufficiently many cipher-

²A decryption oracle could be a smart card, or a server doing RSA decryptions, ...

texts (we are leaving how many these should be for later) and provide them as inputs to the decryption oracle. They record how long it takes for each decryption to finish. As a result, the adversary has a dataset of the form c_i, t_i (c_i being the i -th ciphertext and t_i being the amount of time that was recorded).

The attack recovers the secret exponent bit by bit. We know that the most significant bit is always one. Any other bit can obviously only take the values 0 or 1. Thus for the “next unknown bit” we make a “key bit guess”. This means that the first step in our attack will guess d_{w-1} , which leads to two key candidates: $sk_0(w-1) = (1, 0)$ and $sk_1(w-1) = (1, 1)$. These key guesses enable to model the first few steps of the decryption algorithm. In particular with guessing bit $w-1$ we can predict what happens in the loop when $i = w-1$, and we can also predict what happens in the square at iteration $i = w-2$.

The attack strategy is to guess key bit j in order to predict the behaviour of the square operation in loop iteration $j+1$ the set of ciphertexts. When I write the “behaviour” then I mean any difference in time due to the occurrence (or not) of an extra reduction in the Montgomery multiplication. A simple way of modelling a timing difference is to say use the occurrence of an extra reduction as a way of categorising inputs (and the corresponding timing values). From a procedural point of view the attack that guesses bit j then works as follows:

- For each ciphertext i , use the guess $sk_0(j)$, in order to predict if or not an extra operation occurs in the square operation in the $j+1$ for loop iteration. If an extra operation occurs place timing value t_i of ciphertext i in set s_1 , otherwise place the timing value in set t_0 . Compute the average of each set, and take the difference of averages: $\delta_{j0} = \bar{s}_1 - \bar{s}_0$.
- For each ciphertext i , use the guess $sk_1(j)$, in order to predict if or not an extra operation occurs in the square operation in the $j+1$ for loop iteration. If an extra operation occurs place timing value t_i of ciphertext i in set s_1 , otherwise place the timing value in set t_0 . Compute the average of each set, and take the difference of averages: $\delta_{j1} = \bar{s}_1 - \bar{s}_0$.
- Either guess $sk_0(j)$ corresponds to the correct key or guess $sk_1(j)$ corresponds to the correct key. In order to determine which guess is in fact correct we check what give the bigger value δ_{i0} or δ_{i1} . The bigger value indicates the correct guess.

Why does this work? For a correct guess, the categorisation of timing values is correct for all ciphertexts. For an incorrect guess, the categorisation of timing values is only sometimes correct. Because an extra operation takes extra time, we expect that when we always categorise timing values correctly, we group those timing values together that correspond to needing a bit more time because of the extra operation. Therefore the group s_1 will have a larger average than the group s_0 . When we categorise timing values

only sometimes correctly, then values that correspond to doing an extra step get mixed up with values that correspond to not doing an extra step. Therefore there will be a mix of timing values in both groups, and thus, the averages of the two groups will be closer together.

I will not make this attack more formal or go into it: it just serves as a boilerplate example of how differential input attacks work:

- The adversary knows inputs/outputs, and some (high level) implementation details (algorithm, implementation options, some parameters).
- The Adversary records side channel information during the processing of the inputs/outputs.
- The Adversary predicts some behaviour of the monitored side channel based on knowledge of the inputs/outputs, their implementation knowledge, and a guess about a small part of the secret key. This implies that they do a **divide and conquer** based attack.
- The Adversary uses some statistics to decide which of the key guesses is the one that most likely corresponds to the secret key.

3.2 Power and EM attacks

Power and EM information tend to be sampled at a high frequency and thus we can acquire many measurements values for a single execution of an algorithm (i.e. we get a “trace”). As a consequence it may be possible to observe features that can be directly mapped to the flow of the execution of an algorithm. For instance, in the left-to-right exponentiation that we assume is implemented in a typical RSA decryption, there is a key dependent conditional branch (if and only if a key bit is 1 the multiplication is executed). This feature can be visible in power traces, in which case one can “read off” the secret key from a side channel trace. Obviously such an implementation is completely insecure. So let us consider an implementation that does not give up its secret key via a visual inspection.

In the previous section we used timing information to distinguish key bits and thereby recover the decryption key. The exact same process also works with other types of side channel information such as power or electromagnetic traces. Like before, the adversary would select a set of ciphertexts (whereby the size of the set requires some consideration), and then build a model that predicts some property that is then tested for “consistency” with data obtained from the actual device/oracle under attack.

For instance the property could be the value of a specific bit of the result of squaring in the **next** iteration of the for loop. Based on this prediction the power traces sampled from the ciphertexts can be split into two sets (one where the value of the bit is predicted to be 1 and one where it is predicted to be 0). The difference in average power consumption is our distinguisher as in the timing attack example. We do this for both key bit guesses and thus end up with two difference traces. One of the key bit guesses must be correct and thus at some point in the power trace the adversary correctly predicts (part of) the behaviour of the device and this must result in a better separation between the sets (than in the case of the incorrectly predicted bit).



Figure 2: RSA Correlation traces (taken from Messerges et al., 1999)

We will take a closer look at how such attack are actually carried out, but before this, we will have a quick look at some countermeasures.

4 Countermeasures

Implementations in practice typically deploy a three pronged approach to mitigate side channel attacks. The goal of countermeasures such as *masking* or *hiding* is to remove or at least reduce the dependency between observable side channel information and key-dependent intermediate values. The goal of leakage aware *protocols* or *leakage resilient schemes* is to refresh key material before too much leakage can get into the hands of adversaries.

The countermeasure that we will look at in detail is masking: it is in fact a deployment of secret sharing either as part of a software implementation, or, it may even be deployed at a gate level in practice.

4.1 Hiding

The basic idea of hiding is to remove the data dependency of the leakage. This means that either the execution of the algorithm is randomized or the leakage characteristics of the device are changed in such a way that an attacker cannot easily find a data dependency. The power consumption in particular can be changed in two ways to achieve this goal: the device can be built in such a way that every operation requires approximately the same amount of power, or it can be built in such a way that the power consumption is more or less random. In both cases, the data dependency of the power consumption is reduced significantly. However, in practice the data dependency cannot be removed completely—there always remains a certain amount of data dependency.

Hiding countermeasures are popular in practice, but they have the limitation that they increase the effort of an attacker only **linearly**.

4.2 Masking

The basic idea of masking is to secret share the intermediate values that are processed by the cryptographic device. The motivation behind this approach is that the power that is needed to process shared out (and therefore random looking) intermediate values is independent of the actual intermediate values. A big advantage of masking is that the leakage characteristics of the device do not need to be changed. For instance, the power consumption can still be data dependent. Attacks are prevented because the device processes secret shared (i.e. random looking) intermediate values only.

On a high level, we split each (intermediate) value v into two (or more shares) by generating random values m (the masks) and exclusive oring them with v to generate the secret sharing of v . You should recognise already that this corresponds to the idea of Boolean secret sharing that we touched on before. In the context of practical implementations such as AES, often only two shares are used, i.e. we then have a masked value v_m and the mask m : $v = (v_m, m)$ whereby $v = v_m \oplus m$. The mask is generated internally, it must be drawn at random from a uniform distribution, and it must not be revealed to the adversary. To protect all intermediate values, the default strategy is to mask the AES state as well as the key schedule at the start of encryption (decryption).

Before we can get into some details here, we need to take a brief detour and recall how AES works. Then we can have a look at how to mask it in the way that I just described, and then we can consider why this may not be sufficient for really high-security implementations. All these considerations then set the scene for looking at some masking schemes that have more shares, and that are more interesting to analyse.